

OSSP (Operating Systems and Systems Programming) –25CS2104E

OSSP LAB – SKILL-3

Aim

To implement command-line history management using escape sequences and dynamic memory allocation, including command navigation, input buffer updates, linked-list management, and memory verification using Valgrind.

Algorithm

1. Start the program.
2. Read user commands and store them in a history list.
3. Use escape sequences to navigate previous and next commands.
4. Update the input buffer when a command is recalled.
5. Dynamically allocate and resize memory as required.
6. Manage the history using a linked list.
7. Free allocated memory after use.
8. Verify memory usage and leaks using Valgrind.
9. Stop the program.

Task 1

```
Execution Result = 30  
bhavana@Bhavana:~$ nano history.c
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_LENGTH 100

int main() {
    char *history[MAX_HISTORY];
    char input[MAX_LENGTH];

    int history_count = 0;
    int current = -1;

    printf("Simple Command History\n");
    printf("Type 'exit' to quit.\n");

    while (1) {

        printf("\nmysh> ");

        if (fgets(input, MAX_LENGTH, stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        /* Exit */
        if (strcmp(input, "exit") == 0)
            break;

        /* Ignore empty input */
        if (strlen(input) == 0)
            continue;

        /* Store command */
        if (history_count < MAX_HISTORY) {
            history[history_count] = malloc(strlen(input) + 1);

```

```

            if (history_count < MAX_HISTORY) {
                history[history_count] = malloc(strlen(input) + 1);

                if (history[history_count] == NULL) {
                    printf("Memory allocation failed.\n");
                    return 1;
                }

                strcpy(history[history_count], input);
                history_count++;
            }

            current = history_count;

            printf("Command stored: %s\n", input);
        }

        /* Display history */
        printf("\nCommand History:\n");

        for (int i = 0; i < history_count; i++) {
            printf("%d: %s\n", i + 1, history[i]);
        }

        /* Free memory */
        for (int i = 0; i < history_count; i++) {
            free(history[i]);
        }

        return 0;
    }
}

```

```

bhavana@Bhavana:~$ gcc history.c -o history
bhavana@Bhavana:~$ ./history
Simple Command History
Type 'exit' to quit.

mysh> ls
Command stored: ls

mysh> pwd
Command stored: pwd

mysh> exit

Command History:
1: ls
2: pwd

```

Task 2

```

bhavana@Bhavana:~$ nano memory.c

```

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 2
#define BUFFER_SIZE 100

struct Node {
    char *command;
    struct Node *next;
};

int main() {

    int capacity = INITIAL_SIZE;
    int count = 0;

    char **commands = malloc(capacity * sizeof(char *));

    if (commands == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    char input[BUFFER_SIZE];

    printf("Enter commands.\n");
    printf("Type 'exit' to stop.\n\n");

    while (1) {

        printf("Command: ");

        if (fgets(input, BUFFER_SIZE, stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';
    }
}

```

```

U nano 8.7.1
    if (strcmp(input, "exit") == 0)
        break;

    if (strlen(input) == 0)
        continue;

    /* Resize array when full */
    if (count == capacity) {

        capacity *= 2;

        char **temp = realloc(
            commands,
            capacity * sizeof(char *)
        );

        if (temp == NULL) {
            printf("Reallocation failed.\n");
            break;
        }

        commands = temp;

        printf("Array resized to %d elements.\n", capacity);
    }

    /* Allocate memory for command */
    commands[count] = malloc(strlen(input) + 1);

    if (commands[count] == NULL) {
        printf("Memory allocation failed.\n");
        break;
    }

    strcpy(commands[count], input);

```

```

GNU nano 8.7.1
    }

    printf("\nStored Commands:\n");

    for (int i = 0; i < count; i++) {
        printf("%d: %s\n", i + 1, commands[i]);
    }

    /* Create linked list */
    struct Node *head = NULL;
    struct Node *tail = NULL;

    for (int i = 0; i < count; i++) {

        struct Node *newNode =
            malloc(sizeof(struct Node));

        if (newNode == NULL) {
            printf("Node allocation failed.\n");
            break;
        }

        newNode->command = malloc(
            strlen(commands[i]) + 1
        );

        if (newNode->command == NULL) {
            free(newNode);
            break;
        }

        strcpy(newNode->command, commands[i]);

        newNode->next = NULL;

        if (head == NULL) {
            head = newNode;

```

```

    }

    printf("\nLinked List:\n");

    struct Node *current = head;

    while (current != NULL) {
        printf("%s -> ", current->command);
        current = current->next;
    }

    printf("NULL\n");

    /* Free linked list */
    current = head;

    while (current != NULL) {

        struct Node *temp = current;

        current = current->next;

        free(temp->command);
        free(temp);
    }

    /* Free command array */
    for (int i = 0; i < count; i++) {
        free(commands[i]);
    }

    free(commands);

    printf("\nAll memory released successfully.\n");

    return 0;
}

```

```

bhavana@Bhavana:~$ gcc -Wall -Wextra memory.c -o memory
bhavana@Bhavana:~$ ./memory
Enter commands.
Type 'exit' to stop.

Command: ls
Command: pwd
Command: exit

Stored Commands:
1: ls
2: pwd

Linked List:
ls -> pwd -> NULL

All memory released successfully.

```

Result

Thus, the programs were successfully implemented and tested in Ubuntu to demonstrate command history and escape-sequence handling, along with dynamic memory allocation, array resizing, linked-list management, safe buffer handling, and proper memory deallocation verified using Valgrind.